FULL PROJECT DOCUMENTATION (A TO Z)

1. Executive Overview & Purpose

> **Presentation Evaluator** is an automated, AI-driven assessment system designed to help students,
>
> executives, and public speakers measure, refine, and master their communication and speech delivery
>
> skills.

> In traditional speech preparation, presenters lack objective, quantitative feedback regarding speech
>
> cadence (Words Per Minute), filler word frequency (e.g., 'um', 'uh', 'like'), slide visual structure, and delivery
>
> sentiment. This platform solves this challenge by analyzing uploaded presentation recordings or live
>
> microphone rehearsals and providing instant, data-driven feedback, interactive timeline charts, overall

> score badges, and exportable branded PDF executive reports.

2. Technology Stack Breakdown (A to Z)

The following table provides a comprehensive overview of all frameworks, libraries, tools, and databases utilized across the full-stack architecture and their specific roles in the project:

Technology	Category	Function & Role in Project
React 18	Frontend UI Library	Manages reactive UI rendering, component hierarchy, view switching, and stateful presentation interfaces.
TypeScript (TS)	Programming Language	Enforces strict compile-time type safety across data interfaces (`types/index.ts`), props, and state definitions.
Vite 6	Build Tool & Dev Server	Provides ultra-fast Hot Module Replacement (HMR) during development and generates optimized bundle assets.
Tailwind CSS 3	Styling Framework	Powers the modern design system, glassmorphism cards, responsive layouts, and seamless Light/Dark theme switching.
Recharts	Data Visualization	Renders interactive timeline graphs for Words Per Minute (WPM) cadence tracking and performance metric breakdowns.
jsPDF + html2canvas	PDF Document Engine	Captures HTML report DOM elements and programmatically generates downloadable executive PDF documents on the client side.
Lucide React	UI Icon Library	Provides lightweight, consistent vector icons for navigation hubs, metrics cards, and action controls.
Canvas Confetti	Visual Animations	Triggers celebratory particle animations upon successful rehearsal analysis and report generation.
Java 21	Backend Language	Provides modern, high-performance LTS Java execution runtime for

Technology	Category	Function & Role in Project
		backend business logic.
Spring Boot 3.3	Backend Framework	Powers RESTful Web Services, Dependency Injection, Service Repositories, and Cross-Origin Resource Sharing (CORS).
Spring Data JPA	ORM Framework	Handles Object-Relational Mapping, connecting Java domain entities to database tables with automated SQL execution.
PostgreSQL 16	Relational Database	Stores structured relational data including user accounts, evaluation scores, metadata, and historical records.
pgvector Extension	Vector Database Engine	Stores 1536-dimensional AI vector embeddings for transcript segments and performs native Cosine Similarity Search (`<=>`).

3. System Architecture & Processing Workflow

1. **Stage 1: Input Ingestion**
    - Users upload presentation files (.mp4 video, .wav/.mp3 audio, .pdf/.pptx slides) or record
    > live speech directly via the built-in **Live Mic Recording Hub**.
2. **Stage 2: AI Processing Pipeline**
    - **Neural Audio Ingestion:** Converts speech into structured text transcripts.

- **Cadence Velocity Calculation:** Measures speaking rate in Words Per Minute (WPM) across

  > timestamps.

- **Filler Word & Pause Classification:** Categorizes filler words ('um', 'uh', 'like') and measures

  > hesitation rates.

- **Visual & Slide Scoring:** Evaluates visual balance, readability, and bullet-point density.

3. **Stage 3: Output & Executive Report Synthesis**

   > •Generates overall score badges (e.g., 92% - Top 5% speaker tier), Recharts pace charts,

   > customized strengths and areas of improvement, and downloadable executive PDF reports.

4. Complete Project Directory Structure

```
presentation-evaluator/ ├── src/ # Frontend React Codebase │ ├── components/ # React UI Components │ │ ├── auth/ # LoginForm.tsx, RegisterForm.tsx │ │ ├── dashboard/ # Dashboard.tsx (Upload Hub & Control Panel) │ │ ├── evaluation/ # EvaluationResultView.tsx (Charts & Feedback) │ │ ├── landing/ # LandingPage.tsx │ │ ├── layout/ # Navbar.tsx, Sidebar.tsx, Footer.tsx │ │ ├── profile/ # ProfileDashboardView.tsx, SettingsView.tsx │ │ └── reports/ # ReportPreviewModal.tsx (PDF Generation Engine) │ ├── context/ # AuthContext, EvaluationContext, ThemeContext │ ├── services/ # api.ts (Spring Boot API Connection Client) │ ├── types/ # index.ts (TypeScript Interfaces & Types) │ └── utils/ # mockData.ts ├── backend/ # Backend Java Spring Boot Codebase │ ├── src/main/java/com/presentation/evaluator/ │ │ ├── config/ # CorsConfig.java │ │ ├── controller/ # AuthController, EvaluationController,
```

```
VectorSearchController │   │   ├── dto/ # LoginRequest, RegisterRequest, AnalysisRequest │   │   ├── entity/ # UserEntity, EvaluationEntity, TranscriptSegmentEntity │   │   ├── repository/ # UserRepository, EvaluationRepository, TranscriptSegmentRepository (pgvector) │   │   └── service/ # AuthService, EvaluationService, VectorSearchService │   ├── src/main/resources/ # application.yml, schema.sql (pgvector DDL & HNSW index) │   ├── docker-compose.yml # PostgreSQL 16 + pgvector container configuration │   └── pom.xml # Maven dependencies (Java 21, Spring Boot 3.3) └── package.json # Frontend dependencies
```

5. Execution Commands & Startup Guide

> **1. Starting the Frontend Dev Server (React + Vite):**

```
cd /home/kulayappa/Desktop/SE_Project/presentation-evaluator npm run dev # Server running at: http://localhost:5173
```

> **2. Starting the Backend Server (Java Spring Boot 3):**

```
cd /home/kulayappa/Desktop/SE_Project/presentation-evaluator/backend ./mvnw spring-boot:run # REST API running at: http://localhost:8080/api/v1
```

> **3. Starting PostgreSQL + pgvector Container:**

```
cd /home/kulayappa/Desktop/SE_Project/presentation-evaluator/backend docker compose up -d # Database listening at: localhost:5432 (Database: presentation_db)
```